

Threading in C++ 11

Overview

Creating and deleting threads

Thread local storage

C++ 11 threading API

- Includes both a thread class and supporting classes.
 - Create and run a thread
 - `thread::thread(threadMain, params ...)`
 - Thread function
 - `auto threadMain = [] (parameters) { .... }`
 - Join
 - `thread::join();`
 - Identifier for the thread
 - `thread::get_id();`
 - Header file
 - `#include <thread>`
- Note: This is a small subset of the thread functions

Basic threading

```
1.  #include <iostream>
2.  #include <thread>
3.  using namespace std;

4.  int threadMain (int* i) {
5.      this_thread::sleep_for(chrono::seconds(*i));
6.      cout << "thread " << *i << " sleeping for " << *i << " secs" << endl;
7.      return 0;
8.  };

9.  void main(int, char**) {
10.     const auto maxThreads = 4u;
11.     thread threads[maxThreads];

12.     for (auto i=0; i<maxThreads; i++) {
13.         threads[i] = thread(threadMain, &i);
14.     }

15.     for (auto& thread : threads)
16.         thread.join();
17. }
```

thread 2 sleeping for 2 secs
thread 2 sleeping for 2 secs
thread 3 sleeping for 3 secs
thread 4 sleeping for 4 secs

Basic threading

```
1.  #include <iostream>
2.  #include <thread>
3.  using namespace std;

4.  int threadMain (int* i) {
5.      this_thread::sleep_for(chrono::seconds(*i));
6.      cout << "thread " << *i << " sleeping for " << *i << " secs" << endl;
7.      return 0;
8.  };

9.  void main(int, char**) {
10.     const auto maxThreads = 4u;
11.     int param[maxThreads];
12.     thread threads[maxThreads];

13.     for (auto i=0u; i<maxThreads; i++) {
14.         param[i] = i;
15.         threads[i] = thread(threadMain, &param[i]);
16.     }

17.     for (auto& thread : threads)
18.         thread.join();
19. }
```

thread 0 sleeping for 0 secs
thread 1 sleeping for 1 secs
thread 2 sleeping for 2 secs
thread 3 sleeping for 3 secs

Basic threading, using lambdas

```
1.  #include <iostream>
2.  #include <thread>
3.  using namespace std;

4.  void main(int, char**) {
5.      const auto maxThreads = 4u;
6.      int param[maxThreads];
7.      thread threads[maxThreads];

8.      auto threadMain = [](int* i) {
9.          this_thread::sleep_for(chrono::seconds(*i));
10.         cout << "thread " << *i << " sleeping for " << *i << " secs" << endl;
11.         return 0;
12.     };

13.     for (auto i=0u; i<maxThreads; i++) {
14.         param[i] = i;
15.         threads[i] = thread(threadMain, &param[i]);
16.     }

17.     for (auto& thread : threads)
18.         thread.join();
19. }
```

thread 0 sleeping for 0 secs
thread 1 sleeping for 1 secs
thread 2 sleeping for 2 secs
thread 3 sleeping for 3 secs

Arguments

- Arguments passed to **thread**, **sync** and **call_once** are copied to the thread private area.
- To prevent copying requires the use of a reference wrapper
 - `thread t(functionA, ref(referenceParameter));`

Thread local storage

- Thread local storage is a term that can be applied to variable that alters global or static data to a copy per thread.
- The lifetime is the same as the thread.

```
1. class SharedObject {  
2.     int _objectCounter;  
3.     static int _classCounter;  
4.     thread_local static int _threadLocalCounter;  
5.     ...  
6. };
```

Threading in Win32

Overview

Creating and deleting threads

Win32 threading API

- Create a thread
 - `_beginthread (threadMain, stackSize, argumentList)`
 - `_beginthreadex (security, stackSize, threadMain, argumentList, initFlag, threadId)`
- Terminate a thread
 - `_endthread ( )`
 - `_endthreadex ( )`
- Thread function
 - `void (* threadMain)( void * )`
- Header file
 - `#include <process.h>`
- Note: This is a small subset of the thread functions

Win32 warning !!!

- Do not use `CreateThread()`
 - This is a Win32 not a CRT function, and is not entirely compatible with the CRT
 - You may experience a small memory leak
 - `_beginthread()/_beginthreadex()` call `CreateThread()` but only after executing their own thread initialisation code.
- `_endthread()` closes the handle thread implicitly
- `_endthreadex()` leaves the thread handle open. Close with a call to `CloseHandle()`

Basic threading

```
1.  #include <iostream>
2.  #include <windows.h>
3.  #include <process.h>
4.  using namespace std;

5.  unsigned int __stdcall threadMain(void* pParam){
6.      int i = *((int*)pParam);
7.      Sleep (i*1000);
8.      cout << "thread " << i << " sleeping for " << i << " secs" << endl;
9.      return 0;
10. }

11. void main(int, char**) {
12.     const int maxThreads = 4;
13.     HANDLE threads[maxThreads];

14.     for (int i=0; i<maxThreads; i++)
15.         threads[i] = (HANDLE) _beginthreadex (NULL, 0, threadMain, &i, 0, NULL);

16.     WaitForMultipleObjects (maxThreads, threads, TRUE, INFINITE);
17.     for (int i=0; i<maxThreads; i++)
18.         CloseHandle (threads[i]);
19. }
```

thread 0 sleeping for 0 secs
thread 1 sleeping for 1 secs
thread 2 sleeping for 2 secs
thread 3 sleeping for 3 secs

Waiting for threads

- Wait functions wait until a specified criteria has been met
- WaitForSingleObject returns when one of the following occurs:
 - The specified object is in the signalled state.
 - The time-out interval elapses
- WaitForMultipleObjects can be used as a join statement

Threads, classes and windows

- There are a number of architectures for deploying threads within an OO window environment.
 - Threadmain as static method within MyApp, launched and managed from within MyApp.
 - Threadmain as static method within a subcomponent (eg. Physics class), launched and managed from within MyApp.
 - Abstract Thread class encapsulating both Threadmain as static method and thread management, which is then inherited by a subcomponent, and launched from within MyApp.



Thread class

```
1.  class Thread {
2.  private:
3.      HANDLE _hThread;
4.      bool _terminate;

5.      static unsigned __stdcall threadFunc(void *param) {
6.          if (param)
7.              return ((Thread*)param)->run();
8.          return 1; // Return error
9.      }
10. public:
11.     Thread() : _hThread(0), _terminate(false) {}
12.     virtual ~Thread() {}

13.     void terminate() { _terminate = true; }
14.     bool isTerminated() const { return _terminate; }

15.     HANDLE start();

16.     void waitForTermination();

17.     virtual int run() = 0;
18. };
```

Thread class

```
1.  HANDLE Thread::start() {
2.      unsigned threadId=0;
3.      _hThread = (HANDLE)_beginthreadex(
4.          NULL,                // no security attributes (child cannot inherited handle)
5.          1024*1024,           // 1MB stack size
6.          threadFunc,          // code to run on new thread
7.          this,                // pointer to host application class
8.          0,                   // run immediately (could create suspended)
9.          &threadId            // OUT: returns thread ID
10.     );
11.     return _hThread;
12. }

13. void Thread::waitForTermination() {
14.     WaitForSingleObject(_hThread, INFINITE); // wait for it to stop
15.     CloseHandle(_hThread);                  // close thread handle
16.     _hThread=0;
17. }
```

Thread class, used by MyApp

```
1.  Class Physics: public Thread {  
2.  ...  
3.  };  
  
4.  class MyApp: public App {  
5.      Physics _ballPhys;  
6.  ...  
7.  };  
  
8.  void MyApp::OnCreate() {  
9.      _ballPhys->start();  
10. }  
  
11. void MyApp::OnDestroy() {  
12.     _ballPhys->terminate();  
13.     _ballPhys->waitForTermination();  
14. }
```


Asynchronous functions in C++ 11

Async
Futures

Async function

- The template function `async` runs a function asynchronously (potentially in a separate thread)
- The caller of an `async` function does not wait for it to return before proceeding to the next instruction
- The `async` function returns (completes) at some later stage
- Results from an `async` function can be accessed by using a future object.
- The `get` method within the future object blocks until the data is available

Async example

```
1.  #include <iostream>
2.  #include <string>
3.  #include <future>
4.  using namespace std;

5.  string functionA() {
6.      return "Hello ";
7.  }

8.  string functionB() {
9.      return "World\n";
10. }

11. int main(int, char **) {
12.     //Sync
13.     cout << "Sync Message= ";
14.     cout << functionA();
15.     cout << functionB() << endl;
16.
17.     //Async
18.     {
19.         future<string> f1 = async(functionA);
20.         future<string> f2 = async(functionB);
21.         this_thread::sleep_for(chrono::milliseconds(100));
22.         cout << "Async Message= ";
23.         cout << f1.get();
24.         cout << f2.get() << endl;
25.     }
26.     return 0;
}
```

Deferred (lazy) execution

- The template function `async` prepares to run a function asynchronously (potentially in a separate thread)
- The caller of an `async` function does not wait for it to return before proceeding to the next instruction
- Results from an `async` function can be accessed by using a future object.
- The `async` function runs when the `get` method within the future object blocks attempts to retrieve the data
- The `get` method blocks until the data is available

Deferred execution

```
1.  #include <iostream>
2.  #include <string>
3.  #include <future>
4.  using namespace std;

5.  string functionA() {
6.      return "Hello ";
7.  }

8.  string functionB() {
9.      return "World\n";
10. }

11. int main(int, char **) {
12.     //Sync
13.     cout << "Sync Message= ";
14.     cout << functionA();
15.     cout << functionB() << endl;

16.     //Deferred Async
17.     {
18.         future<string> f1 = async(launch::deferred, functionA);
19.         future<string> f2 = async(launch::deferred, functionB);
20.         this_thread::sleep_for(chrono::milliseconds(100));
21.         cout << "Async Deferred Message= ";
22.         cout << f1.get();
23.         cout << f2.get() << endl;
24.     }
25.     return 0;
26. }
```

Advantages of asynchronous invocation

- Implicitly using threads by using asynchronous invocation has several advantages
 - Environment takes care of creating and managing all threads for us
 - No need for explicit concurrency control operations
 - Improves performance by enabling long-running tasks to be started without stalling application
- Asynchronous invocation not suitable for
 - Code which needs to communicate between threads
 - Applications which need to prioritize threads
 - This requires manual thread creation